



Today's agenda

- ↳ longest increasing subsequence
- ↳ Russian doll envelope
- ↳ building bridges



AlgoPrep



(2) Longest increasing subsequence $\sim O(n^2)$

↳ Given $arr[n]$, return the length of longest increasing subsequence.

Strictly

Ex: $arr[10]: 10 \ 22 \ 9 \ 33 \ 21 \ 50 \ 41 \ 60 \ 80 \ 1$

↳ ans = 6

{ 10 22 33 41 60 80 }

Idea 1

↳ Find all the subsequence and pick the one with longest length and increasing.

T.C: $O(2^n * n)$

Idea 2

$arr[10]: 10 \ 22 \ 9 \ 33 \ 21 \ 50 \ 41 \ 60 \ 80 \ 1$

dp:

1	2	1	3	2	4	3	5	6	1
---	---	---	---	---	---	---	---	---	---

↳ ans = 6



//Pseudo code

T.C: $O(N^2)$
S.C: $O(N)$

```
int LIS ( int arr[N] ) {  
    int dp[N] = {1};  
    int ans = 1;  
    for (int i = 1; i < N; i++) {  
        for (int j = i - 1; j >= 0; j--) {  
            if (arr[i] > arr[j]) {  
                dp[i] = max(dp[i], dp[j] + 1);  
            }  
        }  
        ans = max(ans, dp[i]);  
    }  
    return ans;  
}
```



Q2) Longest increasing subsequence $\rightarrow O(N \log N)$

↳ Given $arr[N]$, return the length of longest increasing subsequence.

Ex: $arr = [10, 22, 9, 33, 21, 50, 41, 60, 80, 32]$

dp:

0	1	2	3	4	5	6	7	8	9
10	22	9	33	21	50	0	0	0	0

ans = 2 8 4 8 6



AlgoPrep



// Pseudo code

```
int LIS (int arr[N]) {
```

```
    int dp[N];  
    int ans = 0;
```

```
    for (int i = 0; i < N; i++) {
```

```
        int lo = 0;
```

```
        int hi = ans;
```

```
        int ans = -1;
```

```
        while (lo < hi) {
```

```
            int m = lo +  $\frac{(hi - lo)}{2}$ ;
```

```
            if (dp[m] < arr[i]) {
```

```
                lo = m + 1;
```

```
            }
```

```
            else { // dp[m] >= arr[i]
```

```
                hi = m;
```

```
            }
```

```
        }
```

```
        dp[lo] = arr[i];
```

```
        if (lo == ans) {
```

```
            ans++;
```

```
        }
```

```
    }
```

```
}
```

```
    return ans;
```

T.C: $O(N \log N)$
S.C: $O(N)$



```
int LIS (int arr[N]) {
```

```
    int dp[N];  
    int ans = 0;
```

```
    for (int i = 0; i < N; i++) {
```

```
        int lo = 0;  
        int hi = ans; // ans = -1;  
        while (lo < hi) {  
            int m = lo + (hi - lo) / 2;  
            if (dp[m] < arr[i]) {  
                lo = m + 1;  
            }  
            else { // dp[m] >= arr[i]  
                hi = m;  
            }  
        }
```

```
        dp[lo] = arr[i];  
        if (lo == ans) {  
            ans++;  
        }
```

```
    }  
    return ans;
```

Ex: 10 22 9 33 21 50 41 60 80 1

dp:

lo	0	1	2	3	4	5	6	7	8	9
hi	0	1	2	3	4	5	6	7	8	9
	10	22	9	33	21	50	41	60	80	1

ans = 0 2 2 2 4

Break till 10:32pm



Q) Russian doll envelope

↳ Given n envelopes, where every envelope is represented by its $[w_i, h_i]$.

one envelope fits into another only if both the width and height of one envelope are greater than the other envelope's width & height.

Return the max envelope you can stack.

Ex: $\{5, 4\}$ $\{6, 4\}$ $\{6, 7\}$ $\{2, 3\}$

$(2, 3) \rightarrow (5, 4) \rightarrow (6, 7) \rightarrow \text{ans} = 3$

Idea

↳ increasing order of two different parameters clubbed together. $\rightarrow$ LIS

$\{5, 4\}$ $\{6, 4\}$ $\{6, 7\}$ $\{2, 3\}$

$\{2, 3\}$ $\{5, 4\}$ $\{6, 7\}$ $\{6, 4\}$

↳ ans = 3.



Steps:

- ① Sort on the basis of any 1 Parameter.
- ② Apply LIS on the other Parameters.

Pseudo Code

```
int maxEnvelope (int [N][2] envelopes) {
```

```
    Pair[] arr = new Pair[N];
```

```
    for (int i = 0; i < N; i++) {
```

```
        arr[i] = new Pair(envelopes[i][0], envelopes[i][1]);
```

```
    }
```

```
    Arrays.sort(arr);
```

```
    int dp[N];
```

```
    int ans = 0;
```

```
    for (int i = 0; i < N; i++) {
```

```
        int lo = 0;
```

```
        int hi = ans;
```

```
        int ans = -1;
```

```
        while (lo < hi) {
```

```
            int m = lo + (hi - lo) / 2;
```

```
            if (dp[m] < arr[i][1]) {
```

```
                lo = m + 1;
```

```
            }
```

```
            else { // dp[m] >= arr[i][1]
```

```
                hi = m;
```

```
            }
```

```
        }
        dp[lo] = arr[i][1];
```

```
        if (lo == ans) {
```

```
            ans++;
```

```
        }
```

```
    }
    return ans;
```

0	5	4
1	6	4
2	6	7
3	2	3

T.C: $O(N \log N)$

S.C: $O(N)$

3



```
Public class Pair implements Comparable<Pair>{
```

```
    int w;
```

```
    int h;
```

```
    Public int compareTo (Pair o){
```

```
        if (this.w != o.w){
```

```
            return this.w - o.w;
```

```
        } else {
```

```
            return o.h - this.h;
```

```
    }
```

```
}
```

```
}
```

{2 2}

{5 3}

{6 7}

{6 4}

→ ans = 4

↓
decreasing order of height when width is same.

↳ if width is same then they should
not come together in the
ans.



Q) Building bridges

↳ you have been given co-ordinates of bridges.
Find max non-crossing bridge that you can build.

Ex:

8

4

3

5

1

2

6

7

1

xx

3

xx

4

5

2

6

xx

7

8

x:

1

2

3

4

5

6

7

8

→ ans = 5

y:

1

2

3

4

5

6

7

8

Idea

u

8

✓

4

↘

3

↘

5

✓

1

↘

2

u

6

↘

7

1

3

4

5

2

6

7

8

→

1

2

3

4

5

6

7

8

2

6

4

3

5

7

8

1

u, ans = 5